

DCT2102 - Redes de Computadores

Programação Cliente-Servidor

Sockets

João Borges

Universidade Federal do Rio Grande do Norte – UFRN
Centro de Ensino Superior do Seridó – CERES
Departamento de Computação e Tecnologia – DCT

10 de novembro de 2021

Agenda

- 1 Introdução
- 2 Arquitetura Cliente-Servidor
- 3 Programação Cliente-Servidor
- 4 Estudo de Caso
- 5 Conclusões

Agenda

1 Introdução

2 Arquitetura Cliente-Servidor

3 Programação Cliente-Servidor

4 Estudo de Caso

5 Conclusões

Contextualização

- Sistemas computacionais
 - Recursos limitados
 - Processamento
 - Armazenamento
 - Etc.
 - Sistemas multiprogramáveis
 - Processos concorrentes
 - Recursos são compartilhados
 - Vários processos competindo por recurso
- Evolução da Internet
 - Distribuição dos recursos

Viabilização dos Sistemas Distribuídos

- Compartilhamento de recursos remotos
- Através de redes de computadores

Motivação

- Características do compartilhamento remoto
 - 1 Mais processos concorrentes
 - A localização não é mais um limite
 - 2 Comunicação por redes de computadores
 - Troca de mensagens na rede
 - 3 Problemas de comunicação
 - Quem fornece e quem utiliza um recurso?
 - Quais são os recursos fornecidos?
 - Como acessar estes recursos?

Motivação

- Características do compartilhamento remoto
 - ① Mais processos concorrentes
 - A localização não é mais um limite
 - ② Comunicação por redes de computadores
 - Troca de mensagens na rede
 - ③ Problemas de comunicação
 - Quem fornece e quem utiliza um recurso?
 - Quais são os recursos fornecidos?
 - Como acessar estes recursos?

Necessidade de uma organização entre os processos distribuídos

Uma arquitetura de software

Motivação

- A arquitetura de software define [Tabenbaum, 2007]¹:
 - O papel de cada processo na comunicação
 - Como eles devem ser organizados
 - Como eles devem interagir

Principal arquitetura proposta

A arquitetura **Cliente-Servidor**

¹Tanenbaum, A. S., van Steen, M. (2007). *Distributed Systems: Principles and Paradigms*. Prentice Hall, 2ª Ed.

Agenda

1 Introdução

2 Arquitetura Cliente-Servidor

3 Programação Cliente-Servidor

4 Estudo de Caso

5 Conclusões

Conceito

Arquitetura Cliente-Servidor

Arquitetura de computadores distribuídos onde um processo cliente interage com um processo servidor, com o intuito de acessar recursos por ele compartilhados. [Coulouris et al, 2005]^a.

^aCoulouris G., Dollimore J.; Kindberg T. (2005). *Distributed Systems: Concepts and Design*. Addison-Wesley, 3^a Ed.

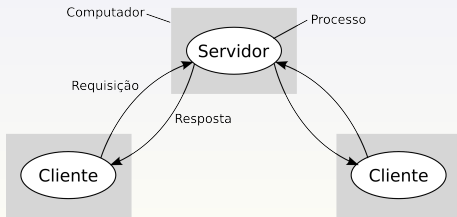


Figura: Arquitetura Cliente-Servidor

Características

- Arquitetura centralizada
- Comportamento de requisição e resposta
 - O cliente requisita um recurso
 - O servidor responde às requisições
- Um servidor para muitos clientes
- Comunicação síncrona ou assíncrona

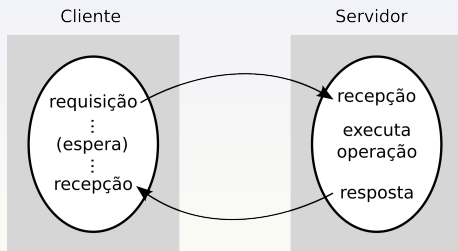


Figura: Comunicação Requisição-Resposta

O Cliente

- É o processo responsável por:
 - Requisitar os serviços remotos
 - Utilizar os recursos remotos
- Funcionamento básico:
 - Realiza requisições a servidores
 - Recebe e trata as respostas das requisições
- Classificação:
 - *Fat-client* (cliente gordo): quando o cliente tem capacidades para não depender do servidor, por ex. jogos web
 - *Thin-client* (cliente magro): quando o cliente tem poucos recursos e depende muito do servidor, ex. Web OS's, tablets.
- Exemplos de cliente:
 - Navegador Web (Firefox)
 - Cliente de e-mail (Outlook)
 - Cliente FTP (CuteFTP)

O Servidor

- É o processo responsável por:
 - Compartilhar os recursos
 - Implementar os serviços
- Funcionamento básico:
 - Aguarda por requisições de clientes
 - Responde a estas requisições
 - Aguarda por novas requisições
- Classificação:
 - Iterativo ou concorrente
 - Serviço centralizado ou distribuído (HTTP ou DNS)
- Exemplos de servidor:
 - Servidor Web (HTTP)
 - Servidor de e-mail (POP3)
 - Servidor de arquivos (FTP)

Vantagens e Desvantagens

Vantagens

- Facilidade de manutenção
- Facilidade de gerenciamento
- Segurança das informações
- Serviços para vários clientes
- Clientes menos robustos

Desvantagens

- Único ponto de falha
- Congestionamento (gargalo)

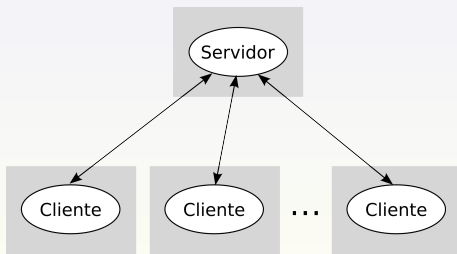


Figura: Vantagens e Desvantagens

Agenda

- 1 Introdução
- 2 Arquitetura Cliente-Servidor
- 3 Programação Cliente-Servidor**
- 4 Estudo de Caso
- 5 Conclusões

Comunicação Entre Processos

- Constitui-se na troca de mensagens entre os processos
- Processos remotos utilizam a rede de computadores
- Implementadas a partir de abstrações das camadas inferiores
- Algumas tecnologias de implementação:
 - RPC (*Remote Procedure Calls* - Chamadas de Procedimentos Remotos)
 - RMI (*Remote Method Invocation* - Invocação Remota de Métodos)
 - CORBA (*Common Object Request Broker Architecture* - Arquitetura Comum para Agente de Requisição de Objetos)
 - **Sockets**

OBS.: Leitura sobre sockets:

Beej's Guide to Network Programming – Using Internet Sockets

<http://beej.us/guide/bgnet/>

Sockets

Conceito

Um canal de comunicação onde uma aplicação pode escrever dados em uma extremidade, que serão transmitidos por uma rede de computadores, e que serão lidos por uma outra aplicação remota. [Tabenbaum, 2007]^a

^aTanenbaum, A. S., van Steen, M. (2007). *Distributed Systems: Principles and Paradigms*. Prentice Hall, 2^a Ed.

- Desenvolvido pelo projeto Berkeley UNIX em 1970
- Abstração/simplificação da camada de transporte
- Padronização da interface com esta camada
- Permite que aplicações acessem os serviços de transporte
- Permite a comunicação por TCP ou UDP

Sockets - System Calls

Tabela: Primitivas de comunicação padronizadas

Primitiva	Significado
SOCKET	Cria um novo canal de comunicação
BIND	Associa um endereço local ao socket
LISTEN	Anuncia a disposição a aceitar conexões
ACCEPT	Bloqueia o processo até receber uma requisição de conexão
CONNECT	Requisição de uma conexão
SEND	Envia dados através do socket
RECEIVE	Recebe dados através do socket
CLOSE	Finaliza a conexão

Sockets - System Calls

// cria um novo socket, retorna o descritor do arquivo ou -1 em erro

```
int socket(int domain, int type, int protocol);
```

// liga um socket a um endereco local (0 - sucesso, -1 erro)

```
int bind(int sockfd, struct sockaddr *my_addr, int addrlen);
```

// escutando por requisicoes de conexao (0 - sucesso, -1 erro)

```
int listen(int sockfd, int backlog);
```

// aceita uma conexao recebida em um bound socket, retorna um novo

// descritor para a conexao estabelecida ou -1 em erro

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

Sockets - System Calls

```
// conecta um socket a um host remoto (0 - sucesso, -1 erro)
int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);

// envia 'len' bytes da msg para o socket,
// retorna a qtd de bytes enviados ou -1 em erro
int send(int sockfd, const void *msg, int len, int flags);

// recebe 'len' bytes do socket ao buffer
// retorna a qtd de bytes recebidos ou -1 em erro
int recv(int sockfd, void *buf, int len, int flags);

// finaliza um socket
close(sockfd);
```

Sockets - Endereçamento do socket

```
// Formato para Internet - IPv4
```

```
struct sockaddr_in
```

```
{
```

```
    short int          sin_family;  // Address family, AF_INET
```

```
    unsigned short int sin_port;    // Port number
```

```
    struct in_addr      sin_addr;    // Internet address
```

```
    unsigned char       sin_zero[8]; // Same size as struct  
                                     // sockaddr
```

```
};
```

```
// Internet address (a structure for historical reasons)
```

```
struct in_addr
```

```
{
```

```
    uint32_t s_addr; // that's a 32-bit int (4 bytes)
```

```
};
```

Sockets - Características TCP e UDP

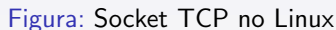
● Características do TCP

- Comunicação orientada a conexão
- Um cliente precisa estabelecer uma conexão
- TCP provê um canal bidirecional confiável
- Os dados são enviados em *stream*
- Dados perdidos são reenviados
- TCP usa controle de fluxo
- Maior atraso na transmissão

● Características do UDP

- Comunicação não-orientada a conexão
- Entrega segundo o melhor esforço
- Os dados são enviados como pacotes
- Pacotes podem se perder
- Pacotes podem ser entregues fora de ordem
- Pacotes entregues com menor atraso e *jitter*

- Comunicação orientada a conexão



Cliente Servidor com Sockets-UDP

- Comunicação não-orientada a conexão

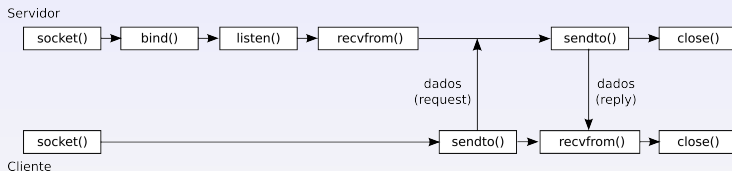


Figura: Socket UDP no Linux

Detalhes de Projeto

- Quais os requisitos da aplicação?
 - Aplicações multimídia requerem pouco atraso e *jitter*
 - Aplicações FTP requerem confiança na entrega dos dados
- O servidor é iterativo ou concorrente?
 - O iterativo atende um cliente por vez
 - O concorrente atende a vários clientes
- A comunicação é síncrona ou assíncrona?
 - O cliente aguardará a resposta do servidor
 - O cliente requisita operações sem aguardá-las

Detalhes de Implementação

- Servidores concorrentes:
 - `fork()`
 - `thread()`
- De acordo com o protocolo da aplicação:
 - *Request* (R) - Requisição
 - *Request-Reply* (RR) - Requisição e resposta
 - *Request-Reply-Acknowledge* (RRA) - Requisição, resposta e confirmação
- De acordo com o protocolo de transporte:
 - Comunicação por TCP insere *overhead*
 - Comunicação por UDP requer cuidados na aplicação

Detalhes de Implementação - Protocolo UDP

Vantagens

- Não possui o *overhead* do TCP
- Confirmações (*ACKs*) não redundantes
 - A mensagem de resposta (*reply*) é a confirmação
- Menos *jitter* nas transmissões

Desvantagens

- A aplicação deve se preocupar com o sucesso das transmissões
- As mensagens devem ser identificadas, para o reenvio
 - Identificador do processo
 - Identificador da requisição
- Tanto cliente quanto servidor podem tratar da comunicação
- Tamanho limitados de dados enviados

Um Servidor de Horas

- Descrição da aplicação:
 - Um servidor que informa a hora certa
 - Um cliente que solicita a hora
 - Baseada em [Ribeiro, 2005]² e [Stevens et al, 2004]³
- Requisitos da aplicação:
 - Conexão confiável
 - Comunicação é síncrona
 - Vários clientes podem acessar o servidor, mas não simultaneamente

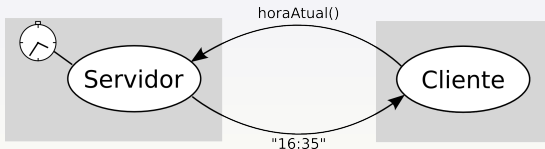


Figura: Servidor de Horas

² Ribeiro, U. (2005). *Sistemas Distribuídos: Desenvolvendo aplicações de alta performance no LINUX*. Axcel

³ Stevens, W. R., Fenner B., Rudoff, A. M. (2004). *Programação de Rede UNIX: API para soquetes de rede*. Bookman

Implementação do Servidor

● Incluindo os cabeçalhos

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <time.h>
```

● Criação do socket

```
int meusocket = socket(AF_INET, SOCK_STREAM, 0);
```

● Definindo endereço do servidor

```
struct sockaddr_in end_cliente;
struct sockaddr_in end_servidor;
end_servidor.sin_family = AF_INET;           // Dominio Internet
end_servidor.sin_addr.s_addr = INADDR_ANY;   // Endereco de rede
end_servidor.sin_port = htons(2343);         // Porta
memset(&(end_servidor.sin_zero), '\0',
      sizeof(end_servidor.sin_zero));        // Zerando estrutura
```

Implementação do Servidor

- Associando o socket a uma porta

```
bind(meusocket, (struct sockaddr *)&end_servidor, sizeof(struct sockaddr));
```

- Habilita escuta de conexões

```
listen(meusocket, 1);
```

- Espera por conexões

```
while(1)
{
    //espera por uma conexao
    int sockettam;
    int socketcon = accept(meusocket, (struct sockaddr *)&end_cliente, &sockettam);
    printf("Conexao estabelecida com %s", inet_ntoa(end_cliente.sin_addr));
    time_t ticks = time(NULL);
    snprintf(buffer, sizeof(buffer), "%.24s ", ctime(&ticks));
    send(socketcon, buffer, strlen(buffer), 0);
    close(socketcon);
}
```

Implementação do Servidor

- Finalizando a conexão

```
close(meusocket);
```

- Compilando e executando o servidor

```
$ gcc servidor.c -o servidor
```

```
$ ./servidor
```

Servidor escutando conexões na porta: 2343

Implementação do Cliente

- Incluindo os cabeçalhos

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
```

- Criando o socket

```
int meusocket = socket(AF_INET, SOCK_STREAM, 0);
```

- Definindo endereço do servidor

```
struct sockaddr_in end_servidor;
end_servidor.sin_family = AF_INET;
end_servidor.sin_addr.s_addr = inet_addr("127.0.0.1");
end_servidor.sin_port = htons(2343);
memset(&(end_servidor.sin_zero), '\0',
    sizeof(end_servidor.sin_zero));
```

// Domínio Internet
// IP do servidor
// Porta do servidor

// Zerando estrutura

Implementação do Cliente

- Conectando ao servidor

```
connect(meusocket, (struct sockaddr *)&end_servidor, sizeof(struct sockaddr));
```

- Recebendo a hora certa

```
char buffer[500 + 1];  
int tamanho = recv(meusocket, buffer, 500, 0);  
printf("Hora certa: %s\n", buffer);
```

- Finalizando a conexão

```
close(meusocket);
```

- Compilando e executando o cliente

```
$ gcc cliente.c -o cliente  
$ ./cliente  
Hora certa: Thu Mar 4 16:35:00 2010
```

Acesso aos códigos

- Disponíveis no seguinte endereço:

`https://github.com/labepi/horacerta`

Resumo

Na aula de hoje vimos:

- Arquitetura Cliente-Servidor
 - Conceito
 - Características
 - Vantagens e desvantagens
- Programação Cliente-Servidor
 - Cliente-Servidor sobre TCP
 - Cliente-Servidor sobre UDP
 - Detalhes de projeto e implementação
- Exemplo de Estudo de Caso
 - Implementação de um Servidor de Horas

Bibliografia

[Coulouris et al., 2005] Coulouris G., Dollimore J.; Kindberg T. (2005). *Distributed Systems: Concepts and Design*. Addison-Wesley, 3ª Ed.

[Tanenbaum, 2007] Tanenbaum, A. S., van Steen, M. (2007). *Distributed Systems: Principles and Paradigms*. Prentice Hall, 2ª Ed.

[Ribeiro, 2005] Ribeiro, U. (2005). *Sistemas Distribuídos: Desenvolvendo aplicações de alta performance no LINUX*. Axcel.

[Stevens et al., 2004] Stevens, W. R., Fenner B., Rudoff, A. M. (2004). *Programação de Rede UNIX: API para soquetes de rede*. Bookman, 3ª Ed.